AGENTS.md

> Guidance for automated coding agents working on `stylometric-transfer`

This document describes the **purpose, architecture, conventions, constraints, and development roadmap** for the `stylometric-transfer` project so that an automated agent can reliably continue development without prior conversation context.

---

1. Project Mission

`stylometric-transfer` implements:

- **Stylometric profiling** — extracting an explicit, interpretable style model from an author's writing corpus
- **Author-conditioned style transfer** — rewriting text to match that style while preserving meaning

The defining design principle is:

> **Explicit, interpretable, versionable style models** rather than fine-tuning or opaque embeddings.

The system must:

- Remain inspectable by humans
- Keep the style model as JSON
- Avoid fine-tuning or hidden representations
- Preserve meaning strictly in rewriting

---

2. Conceptual Terminology (Canonical)

Use the following terms consistently in code, docs, and comments:

- **Stylometry** — quantitative analysis of writing style
- **Stylometric profile / style fingerprint** — the JSON artifact
- **Style transfer** — rewriting while preserving semantics
- **Author-conditioned generation** — generation guided by fingerprint
- **Explicit style model** — rule- and feature-based JSON

Avoid ambiguous terms like "clone" in public documentation.

---

3. Current Architecture

Files

- `fingerprint_style.py`

    - Input: compressed corpus archive (`.zip`, `.tar*`)
    - Output: `style_fingerprint.json`
    - Responsibilities:
        - Extract archive
        - Read text files
        - Compute stylometric measurements locally
        - Filter out blockquotes, reference sections, footnotes, inline citation markers, and boilerplate notices (copyright/terms/privacy) from style analysis
        - Detect fiction vs non-fiction; in non-fiction, multi-word quotations are excluded from profiling (override with `--fiction` / `--non-fiction`)
        - Strip embedded BASE64 images from prompts
        - Select representative excerpts
        - Call LLM to synthesise fingerprint JSON
        - If chunking excerpts, merge partial fingerprints via a dedicated LLM merge prompt
        - Prefilter likely proper-name phrases (honorifics + capitalization-ratio heuristics) and rank phrase candidates with the LLM to drop proper names
        - Optionally rank rare-word candidates with the LLM (shared with common-phrase validation) to de-prioritize proper names
        - Validate common phrases via a separate LLM pass to remove OCR/citation noise (with deterministic prefilters for proper names, entity blacklist matches, and date patterns; disable with `--no-phrase-validation`)
        - Repair invalid JSON if necessary
        - Normalize verbose/duplicative `controls.rewrite_policy` clauses and filter `priority_order` to short tokens before writing the fingerprint
        - Normalize lexicon spelling to a US baseline (except literal hard avoids)
    - CLI short flags: `-c` (config, optional; defaults to `./config.llm.json` if present, else next to script), `-a` (archive), `-o` (out), `-v` (verbose)
    - Extra: `--max-prompt-tokens` overrides chunking threshold
    - Defaults: if `--profile-id` or `--author-name` are omitted, both default to the output filename without the `.json` extension

- `apply_fingerprint.py`
    - Input: fingerprint JSON + Markdown file
    - Output: rewritten Markdown + deviations report
    - Responsibilities:
        - Measure input text
        - Detect fiction vs non-fiction; in non-fiction, multi-word quotations are preserved verbatim (override with `--fiction` / `--non-fiction`)
        - Preserve blockquotes, reference sections, footnotes, and inline citations verbatim (excluded from style transfer)
        - Strip embedded BASE64 images before prompt and re-insert after rewrite
        - Call LLM with fingerprint + measurements
        - Enforce preservation of meaning

        - Score style compliance locally and retry with delta feedback (disable with `--no-style-retry`)
        - Apply `general-guidelines.md` humanizer rules when available, using an LLM parser by default then deterministically filtering out conflicts (disable with `--no-humanizer-llm-parse` or `--no-humanizer-guidelines`)
        - Cache parsed humanizer rules in `humanizer_rules.cache.json` (script directory) and re-parse only when guidelines change
        - Sanitize trailing parenthetical/comma qualifiers in headings when enabled (humanizer_mandatory)
        - Apply heading-case normalization according to `humanizer_mandatory.heading_case_normalization`:
            - `automatic` (no deterministic heading-case transform)
            - `identical` (restore source heading case)
            - `by-level` (per-level policy via `heading_case_by_level`)
        - Optionally preserve source proper-name casing in deterministic heading transforms (`preserve_proper_name_case`)
        - Normalize verbose/duplicative `controls.rewrite_policy` clauses and filter `priority_order` when loading a fingerprint
        - Normalize lexical avoidance checks to US spelling for matching, then apply local spelling to final output
        - Return rewritten text and deviations
        - In verbose mode, report per-chunk attempt scores and when best-attempt selection overrides the last attempt
    - CLI short flags: `-c` (config, optional; defaults to `./config.llm.json` if present, else next to script), `-f` (fingerprint; adds `.json` if missing), `-i` (input), `-o` (out), `-v` (verbose)
    - Extra: `--max-prompt-tokens` overrides chunking threshold
    - Overrides: `--1st-person` / `--2nd-person` / `--3rd-person` force narrative voice regardless of fingerprint
    - Runtime overrides: `--local-spelling {none|canadian|australian|british|us}` (applies to both LLM + deterministic rules), `--local-spelling-llm ...`, `--local-spelling-rules ...`, and `--seed [N]` (omitted value or `0` => random run seed)

- `scripts/fingerprint_style.sh`, `scripts/apply_fingerprint.sh`, and `scripts/show_fingerprint.sh`
    - Bash wrappers around the Python entry points
    - Pass all CLI args through unchanged

- `show_fingerprint.py`
    - Generates a standalone HTML dashboard for a fingerprint JSON

- `utils.py`
    - Shared helper module for lightweight stats and text-splitting primitives used by both entry points

- `prompts.json`
    - Externalized prompt templates used by both Python entry points
    - Located next to the Python scripts; loaded at runtime
- `config.llm.json`
    - Stores API configuration
    - OpenAI-compatible
    - `max_prompt_tokens` controls chunking for large prompts (defaults to `max_tokens`)
    - `max_retries` / `backoff_*` control exponential backoff on transient LLM failures (transport-level retries per HTTP call)
    - Optional `lexicon_hints.json` can be used to inject preferred/avoided phrases into fingerprinting
- `config.tunables.json`
    - Overrides humanizer conflict thresholds for `apply_fingerprint.py`
    - See README for per-field explanations and defaults
    - Includes mandatory humanizer controls (em-dash ban, emoji policy, heading qualifier sanitization, heading-case normalization mode)
    - Includes bounded stochastic variance controls (seeded micro-operations)
    - Includes style retry controls (delta-feedback retries) with separate voice/style caps in forced-person mode
    - `style_retry.max_retries` caps style-loop retries; `style_retry.voice_max_retries` (optional) independently caps voice-loop retries in forced-person mode
    - `humanization_controller.max_feedback_retries` only caps how many style retries include controller-overlay feedback; it does not increase retry count
    - Includes humanization metric weights for the aggregate 0–100 score
    - Includes corpus-derived humanization baseline settings (rolling windows embedded in fingerprints for auditability; stripped from what the LLM sees during rewriting)
    - Includes humanization controller overlays (per-chunk target variation derived from baseline)
    - Includes chunking caps (max input tokens per chunk), split strategy (`chunk_split_on`), and rolling chunk summaries for semantic continuity
    - Includes minimum chunk counts when perturbations are enabled
    - Includes chunk-recovery split controls when the LLM repeatedly returns invalid output
    - Includes variance-aware chunk sizing (smaller chunks for higher-variance styles)
    - Includes lexical signal limits (rare word list size)
    - Includes lexical avoidance limits (rare word list size)
    - Includes controls normalization (rewrite-policy de-dup + priority-order filtering)
    - Includes fiction-detection thresholds (quote span/ratio heuristics)
    - Includes section-restore controls (fuzzy heading match + restoration caps)
    - Includes deterministic redundancy post-processing controls (near-duplicate prose pruning and long unordered-list density throttling)
    - Includes optional sanity checks such as line/word/paragraph count change warnings

    - The current tunables may be embedded into fingerprints as `metadata.extraction.tunables_snapshot` for auditability

- `config.avoid.txt`
    - Optional global avoid list (one word/phrase per line)
    - Applied during fingerprinting (via lexicon hints) and merged into `lexicon.avoid_words` (hard avoids) during application
    - Entries are literal (no spelling normalization); include any local spelling variants you need
- `config.common_words.txt`
    - Optional common-word list (one word per line)
    - Used to derive `measurements.lexical_avoidance.rare_words` (common words absent from the corpus)
    - Supports `word <zipf_frequency>` format to prioritize higher-frequency words
    - These omissions are intended as soft avoids (`lexicon.avoid_words_soft`) rather than hard bans
- `config.entity_blacklist.txt`
    - Optional entity blacklist (people, places, organizations; one per line)
    - Used to suppress proper-name phrases during common-phrase validation
- `config.local_spelling_rules.json`
    - Locale spelling rules used by deterministic local-spelling enforcement in `apply_fingerprint.py`
    - Supports direct variants, suffix variants, and context-aware variants with rule precedence

Data Flow

```
Corpus → Voice-filtered local stats → LLM synthesis → Fingerprint JSON
Fingerprint + Draft → Voice-filtered local stats → LLM rewrite → Styled
Markdown
```

3.1 Fast Onboarding for Agents

If you are joining this codebase cold, do this first:

1. Run `./tests/run_smoke.sh` to validate non-LLM regression health.
2. Read `config.tunables.json` to understand active runtime behavior (many defaults are intentionally overridden there).
3. Confirm active CLI surfaces:
    - `python fingerprint_style.py --help`
    - `python apply_fingerprint.py --help`
    - `python show_fingerprint.py --help`
4. Before changing rewrite behavior, inspect these hotspots:

    - Retry/chunk orchestration and compliance loops in `apply_fingerprint.py`
    - Deterministic post-processing (spelling, quotes, heading normalization) in `apply_fingerprint.py`
    - Lexical normalization and rare/common-word filtering in `fingerprint_style.py`
5. If a change affects tunables or CLI behavior, update all of:
    - `README.md`
    - `AGENTS.md`
    - `config.tunables.schema.json`
    - `UML-Apply.md` / `UML-Fingerprint.md` (if flow changed)

Common pitfalls:

- Retry counts are **additional passes** after attempt 1 (`max_retries=2` means up to 3 total attempts in that loop).
- Voice and style retry budgets are separate in forced-person mode; do not assume one shared budget.
- `humanization_controller.max_feedback_retries` only gates controller feedback injection, not total retry count.
- Soft lexical-avoid matching is US-normalized for comparison; final output spelling is localized afterward.

---

4. Design Constraints (Critical)

These must always be preserved.

4.1 Explicit Models Only

- The style model MUST remain:
    - JSON
    - Human readable
    - Editable
    - Versionable

Do NOT introduce:

- Fine-tuning
- Embedding-only representations
- Hidden latent style vectors without interpretation

4.2 Meaning Preservation

In rewriting:

- No new facts
- No new claims
- No new examples
- No semantic drift

Deviation reporting is mandatory when conflicts occur.

4.3 Interpretable Metrics

Local measurements must:

- Remain simple and explainable
- Use distributions over single averages where possible
- Prefer sentence/paragraph/lexical statistics

Avoid black-box scoring models unless clearly labeled optional.

---

5. Style Fingerprint Schema (High-Level)

The fingerprint JSON contains these top-level keys:

- `schema_version`
- `profile_id`
- `metadata`
- `measurements` (verbatim local statistics)
- `targets` (style constraints)
- `lexicon` (preferred / avoided words & phrases)
- `templates` (syntactic + rhetorical moves)
- `controls` (priority + strictness + rewrite policy)
- `validators` (scoring weights + checks)
- `derived_instructions` (compiled prompts)

The schema is **extensible**, but backward compatibility should be preserved when possible.

Note: `metadata.corpus` includes `document_count` and `documents` (per-document metadata list, including title when available).

---

6. Prompting Strategy (Canonical)

6.1 Fingerprint Construction

Process:

1. Filter out non-author voice content (blockquotes, references/footnotes, inline citations)
2. Compute local measurements
3. Validate common phrases with an LLM pass to remove OCR/citation noise (disable with `--no-phrase-validation`)
4. Select representative excerpts
5. Include optional `lexicon_hints.json` if present
6. Send both to LLM with:

    - Schema hint
    - JSON-only requirement
    - Controlled vocabulary instructions

LLM must:

- Embed measurements verbatim
- Infer stylistic targets from measurements + excerpts
- Produce derived instructions for reuse

6.2 Application / Rewriting

Prompt must:

- Include full fingerprint JSON
- Include local measurements of input (computed on author-voice text only)
- Enforce:
    - Preservation of meaning
    - Markdown validity
    - Priority order in constraints

LLM output format is JSON:

```
{
  "final_markdown": "...",
  "deviations": [...],
  "self_check": {...}
}
```

---

7. Coding Conventions

7.1 API Client

- Use OpenAI-compatible REST calls
- Prefer `/v1/chat/completions` unless migrated
- Centralize retry/backoff logic

Recommended improvements (future work):

- Exponential backoff for 429 / 5xx
- Streaming optional flag

7.2 Error Handling

Always handle:

- Invalid JSON output
- Partial JSON
- Code-fenced JSON

Current strategy:

- Attempt strict parse
- If failure, call LLM repair mode

Preserve this pattern.

---

8. Measurement Layer (Do Not Over-Engineer)

Current measurements include:

- Sentence length histogram
- Paragraph length histogram
- One-sentence paragraph rate
- Punctuation rates per 1000 words
- Contraction rate
- US vs Canadian spelling heuristic (English-only)
- Dash / ellipsis counts
- Frequent bigrams / trigrams
- Function-word profile
- Rare-word signals (words the author rarely uses)
- Lexical avoidance list derived from a built-in common-words set (common words the author rarely uses)
- Stance signals (hedging/boosting/pronouns)
- Sentence-opener and transition templates
- Rhetorical move signals (claim/evidence/counterpoint/concession/synthesis)
- Paragraph cadence (opening/closing sentence length stats)
- Epistemic profile (speculative/probabilistic/assertive/directive)
- Syntax texture (subordinate/parenthetical/appositive rates)
- Discourse marker position (start vs mid-sentence rates)
- Repetition signals (bigram/trigram repeat rates)

Schema note:

- `measurements.orthography_signals.spelling_variant` records the spelling heuristic output
- `targets.persona.pronoun_preferences` records preferred pronoun sets (if provided)

Guidelines:

- Prefer approximate signals over fragile exact metrics
- Avoid heavy NLP pipelines unless optional
- Do NOT require spaCy / transformers by default

- Exclude non-author voice regions (blockquotes, references/footnotes, inline citations) from measurements and excerpts

Future additions allowed:

- POS tag distributions (optional)
- Readability indices
- Function word profiles

---

9. Ethics & Safeguards

Always preserve:

- Intended use: personal writing, self-modeling, editing assistance
- Avoid impersonation of living authors

Controls in fingerprint:

- `do_not_imitate_living_author = true`
- `copyright_sensitive = true`

Agents must NOT add features that:

- Automate impersonation
- Mask authorship
- Remove deviation reporting

---

10. Roadmap (Agent Guidance)

High-value next steps, in priority order:

Phase 1 — Reliability

- [x] Add retry + exponential backoff wrapper for API calls
- [x] Add request timeout handling
- [x] Add logging verbosity flag

Phase 2 — Validation

- [ ] Align/centralize fingerprint schema location and naming (current schema file exists as `style_fingerprint_schema.json`)
- [ ] Validate fingerprint outputs against schema in runtime and CI
- [ ] Emit warnings on missing or null critical fields

Phase 3 — Scoring & Feedback

- [x] Add post-rewrite scoring against fingerprint
- [x] Compute divergence metrics (sentence length, punctuation)
- [x] Emit compliance score (0–1)
- [ ] Add per-chunk compliance reason breakdowns (top failed constraints) for operator diagnostics

Phase 4 — Tooling

- [ ] Batch rewrite mode
- [ ] Batch fingerprint mode
- [ ] Diff visualizer (original vs styled)
- [ ] CLI subcommands (`stylometric fingerprint`, `stylometric apply`)

Phase 5 — Generation

- [ ] Add `generate_from_fingerprint.py`
- [ ] Support outlines / prompts conditioned on fingerprint

Refactoring (Future)

- [ ] Consolidate additional shared helpers (masking/placeholder, regex utilities, LLM client wrappers) into a shared module alongside `utils.py` to reduce duplication.

---

11. Testing Strategy

Current state: lightweight regression tests + smoke test

Recommended additions:

- Golden corpus test:
  - Run fingerprint on fixed small corpus
  - Verify stable JSON fields
- Rewrite invariants:
  - Word count within tolerance
  - Named entities preserved
  - No added sentences unless allowed
- JSON validity tests

Current smoke test:

- `tests/run_smoke.sh` — regression suites by default (no LLM calls); end-to-end pipeline + LLM connectivity check when `--llm-tests` is provided (requires valid `config.llm.json`)

Regression suite:

- `tests/test_v1_1_0_regression.py` (run via `./tests/run_v1_1_0_regression.sh`; executed automatically by `run_smoke.sh`)
- `tests/test_v1_5_X_regression.py` (run via `./tests/run_v1_5_X_regression.sh`; executed automatically by `run_smoke.sh`)
- `tests/test_v1_7_X_regression.py` (run via `./tests/run_v1_7_X_regression.sh`; executed automatically by `run_smoke.sh`)

---

12. Naming & Public Interface

Preferred public names:

- Project: `stylometric-transfer`
- Artifact: **Style Fingerprint** or **Stylometric Profile**
- Process: **Stylometric profiling** + **Author style transfer**

Avoid marketing terms like:

- “Magic”
- “Clone”
- “Deep copy”

---

13. Summary for Agents

If you are an automated agent continuing this project:

You are building an **interpretable stylometric profiling and style-transfer system** with these invariants:

- Explicit JSON style models
- Local statistical grounding
- LLM used only for synthesis and rewriting
- Meaning preservation is sacred
- Deviations must be reported
- Human inspectability is a core feature

Any new feature should improve:

- Reliability
- Interpretability
- Reproducibility
- Control

—not raw imitation fidelity alone.

AGENTS.md — operational context for stylometric-transfer